OOP and Design Patterns in SuperMario

Scope, audience, and how to read

This document is a quick entry point for people who want to understand **why** the code is organized the way it is and which classes an in-game operation passes through. The content describes the implementation that exists in the repository; wherever something is a design benefit or an inference, it is called out explicitly — this is not a promise that every class is textbook-perfect.

Read in the following order if you are new to the codebase:

1. **Mental model** to learn the main objects and who owns them.
2. **OOP** so you can read interfaces, subclasses, and `unique_ptr` without confusing the relationships.
3. **The five patterns** as needed: global scope (Singleton), input (Command), notifications (Observer), screens/character state (State), entity creation (Factory Method).
4. **Workflow** to join the pieces into one complete game frame.
5. **Extension recipes**, then check the **pitfall/glossary** before editing code.

The full diagrams live in the class diagram (https://github.com/baoduong2342007/CS202-Group04-FinalProject/blob/main/03_Report/class_diagram.md). Summary-style pattern descriptions live in design patterns (https://github.com/baoduong2342007/CS202-Group04-FinalProject/blob/main/03_Report/design_patterns.md); the present document explains things more deeply using real flows and limits verified against the source.

The one-minute mental model

- `Game` runs the window loop. Each frame it hands event/input/update/render to `GameManager`.
- `GameManager` is a Singleton (a single globally accessible instance) and holds a stack of `std::unique_ptr<IGameState>`. The top state receives input/update; overlay states can be rendered on top of the state below.
- The other shared services are also Singletons per the source: `EventBus` publishes events, `SoundManager` manages audio, and `TextureManager` caches textures. `SaveManager` is the exception: it is an object value-owned by `GameManager`.
- `PlayState` is the gameplay state. It **owns** `Level` and `HUD`, while `Level` owns the Box2D world, Mario, and a list of `std::unique_ptr<Entity>`.
- `Entity` is the abstraction (the shared contract) for objects in the world. The main inheritance chains are `Entity -> Character -> Mario/Enemy` and `Entity -> Item -> Coin/Mushroom/FireFlower/Star`.
- `InputHandler` turns keys into `ICommand` objects. `EventBus` publishes `GameEvent`s to `PlayState`, `HUD`, and `SoundManager` so publishers never call each subscriber directly.
- `EntityFactory` accepts a request of type `EnemyType`, `ItemType`, or a tile code and returns `std::unique_ptr<Entity>`; `Level` is the final owner of the entity.

> You can think of `PlayState` as the director of the level, `Level` as the stage that owns the actors, `Command` as an action request slip, `EventBus` as the announcement loudspeaker, and `State` as the rule set currently in force. This analogy is a reading aid, not a set of class names in the source.

1. Minimum C++ vocabulary

- A **class** is a blueprint; an **object/instance** is a concrete value of a class. For example `PlayState` is a class and `std::make_unique<PlayState>(...)` creates an object.
- A **method** is a member function of an object. `Level::update(float)` is a method.
- An **interface/abstract class** is a class that only states the contract to fulfill. In C++, a method with `= 0` is **pure virtual**; a class that still has a pure virtual method cannot be instantiated directly.
- **`virtual` / `override`** lets a base-class pointer/reference invoke the subclass implementation. This is runtime polymorphism.
- **`std::unique_ptr<T>`** is a smart pointer that solely owns one `T`. When the pointer is destroyed or erased from a container, `T`'s destructor runs. Use `std::move` to transfer ownership; it cannot be copied.
- **Composition** is an object holding other objects to build bigger behavior; **ownership** answers "who is responsible for destruction?". **Aggregation** usually just means holding a non-owning reference/pointer. The source uses all three forms, so do not label every arrow "inheritance".
- **RAII** (*Resource Acquisition Is Initialization*) means a resource's lifetime is tied to an object's lifetime. Here `unique_ptr` manages objects and `Subscription` manages EventBus registrations.

2. The four OOP pillars in the real code

2.1 Encapsulation

Encapsulation hides data and invariants inside a class and exposes only the necessary operations. `Entity` keeps `m_body`, position, animation, and lifecycle flags in the `protected` section; `Mario` keeps power/lives/timers private and only lets callers use methods such as `powerUp`, `powerDown`, `tryStartFireBallShot`. `Level` keeps `m_entities`, `m_world`, `m_mario` private and provides `getEntities()` as a read-only `EntityView`.

A runtime example: input code never edits Mario's Box2D velocity directly. A command calls `Mario::moveRight()` or sets an intent; only `Mario::update/preparePhysics` applies the physics invariants. Similarly, `PlayState` never `delete`s an entity itself — it hands the `unique_ptr` to `Level`.

Limits to remember: some getters return non-owning raw pointers (e.g. `Level::getMario()`), and `Entity` still has `isEnemy()/isItem()` helpers for compatibility. Callers must respect the owner's lifetime; encapsulation does not magically turn a raw pointer into a smart pointer.

2.2 Abstraction

Abstraction keeps what the client needs to know and drops implementation detail. Representative contracts:

- `Entity::update(float)` and `getType()` describe a world object without requiring `Level` to know each Goomba or Coin algorithm.
- `IGameState` describes lifecycle + `processInput/update/render`; `GameManager` does not need to know how drawing Menu differs from Play.
- `ICommand::execute`, `IObserver::onNotify`, `EntityCreator::create` are seams whose implementations can be swapped.

Abstraction does not mean "everything must be an interface". `Level` does not implement an `ILevel`; it is a concrete orchestrator owning many subsystems. Extracting an interface only pays off with multiple implementations or when a clear boundary is needed.

2.3 Inheritance

Inheritance expresses an *is-a* relationship and reuses shared contracts/functionality:

```
Entity
|-- Character
|   |-- Mario
|   `-- Enemy
|       `-- Goomba, Koopa, ...
`-- Item
    `-- Coin, Mushroom, FireFlower, Star
```

`Character` adds health, facing, grounded state; `Enemy` adds the patrol/stomp contract; `Item` adds `onCollect(Mario&)`. Concrete classes provide their own behavior. This is inheritance, not ownership: `Mario : Character` does not mean Mario owns a separate Character.

2.4 Polymorphism

Runtime polymorphism means one interface whose calls are resolved by the actual object at runtime. `Level` stores `std::vector<std::unique_ptr<Entity>>`; the loop calls `entity->update(dt)` and C++ dispatches to Goomba, Coin, FireBall, etc. Likewise, `GameManager` stores `unique_ptr<IGameState>`, `InputHandler` stores `unique_ptr<ICommand>`, and `EventBus` calls `IObserver::onNotify`.

Polymorphism is only safe when the base destructor is virtual; the main bases all have virtual destructors (`Entity`, `IGameState`, `ICommand`, `IObserver`). When a specific API is needed (e.g. `Enemy::setTileMap`), the current code uses `EntityType/EntitySubtype` identity plus conditional casts; that is a cue to read the invariants, not a license to cast freely.

3. Composition, ownership, and RAII

The table below answers "which object contains which?" and clearly separates that from the inheritance tree:

Owner	Component/lifetime	Runtime meaning
`GameManager`	`vector<unique_ptr<IGameState>> m_stateStack`	The manager destroys states on `CHANGE`, `POP`, or stack teardown.
`GameManager`	`SaveManager m_saveManager` (value object)	SaveManager lives as long as the manager but is not a Singleton.
`PlayState`	`unique_ptr<Level> m_level`, `unique_ptr<HUD> m_hud`	Reloading a level creates new ownership; HUD is created after Mario.
`Level`	`unique_ptr<b2World>`, `unique_ptr<Mario>`, `vector<unique_ptr<Entity>>`	Level is the sole owner of the world/entity list and receives child spawns via an outbox.
`Mario`	`unique_ptr<IMarioState> m_statePattern`	The power-up state is replaced when the form changes.
`Entity`	`unique_ptr<AnimationSystem>`	Animation state is cleaned up with the entity.
`PlayState` / `HUD` / `SoundManager`	`vector<Subscription>`	Destroying a token unsubscribes from EventBus via RAII.

Members such as `TextureManager&` inside `Level` or `Mario*` inside a command are non-owning references/pointers. The owner must outlive the consumer. Because reloading a `Level` invalidates the old Mario address, `PlayState::rebindCommands()` is called after load/reload to build commands pointing at the new Mario.

4. SOLID: read it as a checklist, not absolute praise

SOLID is a set of five heuristics (guiding principles) for reducing coupling, not five patterns the project must score 100% on. Current evidence shows the following level of support:

Principle	Evidence in the repository	Bounded conclusion
S — Single Responsibility: a class should have one reason to change	`InputHandler` handles binding/dispatch; `EventBus` handles subscription/notify; `SaveManager` handles save read/write.	Good separation at these boundaries, but `PlayState` and `Level` remain large orchestrators (input, transitions, gameplay/render), so do not claim perfect SRP.
O — Open/Closed: extend through seams, limit client edits	`ICommand` for new commands; `EntityCreator` for new creator families; `EntityFactory` returns the `Entity` base.	A new enemy/item still requires adding an enum and a `switch` case in a creator; `SpawnRequest` is a closed `variant`. Extensible, but not fully closed against mapping edits.
L — Liskov Substitution: subclasses usable wherever the base is	`Level` runs every `unique_ptr<Entity>` through `update`; states run through `IGameState`; enemies/items keep the virtual contracts.	True for the shared contract, but the code still uses subtype/capability checks and casts when specific policy is needed. Do not assume every base method is equally meaningful in every subclass.
I — Interface Segregation: small interfaces, clients not forced to depend on extra methods	`ICommand::execute`, `IObserver::onNotify`, `IMarioState` split by role; `IGameState` has only lifecycle/frame APIs.	These are useful narrow boundaries. In contrast `Entity` carries physics, render, identity, collision, and spawn hooks; that interface is wide because of current needs.
D — Dependency Inversion: depend on abstractions, not concretions	`GameManager` holds `unique_ptr<IGameState>`, EventBus calls `IObserver`, the factory calls `EntityCreator`.	`PlayState` still includes concrete commands/states; `Level` includes concrete entities and `GameManager` reaches Singletons. DIP is applied selectively, not as system-wide dependency injection.

When reviewing a change, ask "which seam must stay stable?" instead of adding interfaces just to spell SOLID. If you truly need to swap physics or the EventBus, that is the moment to consider injection; do not casually change a public contract during a small entity task.

5. The Singleton pattern — one access point for shared services

Meaning and the real participants

Singleton guarantees a class has one instance created/obtained through a shared access point, usually `getInstance()`. In C++, the current implementation uses a function-local `static` (Meyer's Singleton), a private constructor, and deleted copies so callers cannot create a second one.

Role	Evidence and usage in the project
Game coordinator	`GameManager::getInstance()` ; private constructor, deleted copies; manages the state stack and deferred operations. `Game` /states call `changeState` , `pushState` , `update` , `render` through this instance.
Global subject	`EventBus::getInstance()` ; private constructor/destructor and deleted copies; keeps subscription state then notifies observers.
Audio service	`SoundManager::getInstance()` ; copy and move deleted; `Game` initializes the service, states/entities call play/pause music or SFX. It is simultaneously an `IObserver` of EventBus.
Resource cache	`TextureManager::getInstance()` ; private constructor, deleted copies; `Level` takes a reference in `Level::Level()` then passes a non-owning pointer/reference to entities/renderers.

The simple workflow:

1. A caller invokes `X::getInstance()` . The first time, the function-local `static X instance` is constructed; later calls return the same object.
2. `Game::Game()` obtains `SoundManager` and `GameManager` ; `Level::Level()` obtains `TextureManager` ; gameplay publishes events via `EventBus` . No client class `new` s an extra manager next to these instances.
3. The Singleton holds a service/cache or coordination point that must live long enough; gameplay objects still have their own ownership. For example `Level` owns `unique_ptr<Entity>` but does not own `TextureManager` .

Benefits, trade-offs, and limits

- One audio mixer, event registry, texture cache, and state coordinator avoids duplicated resources or two places driving the flow. This is a design benefit inferred from how callers use the services, not a guarantee that every global problem disappears.
- In exchange, `getInstance()` is a global access point: dependencies are hidden, tests struggle to substitute a fake instance, tests easily depend on static order/lifetime, and callbacks/events can create implicit coupling. When a class only needs textures/audio, prefer an explicit non-owning reference, as when `Level` passes the texture manager to entities.
- Singleton does not mean "every shared object". `EntityFactory` has a public constructor; `defaultFactory()` in the compatibility shim is just a function-local helper and does not turn the class into a Singleton.
- **SaveManager is not a Singleton:** `SaveManager` has a public constructor and `GameManager` holds `SaveManager m_saveManager` by value. To read the save use `GameManager::getInstance().getSaveManager()` ; do not add `SaveManager::getInstance()` .

Evidence: `include/core/GameManager.h:19-50` , `src/core/GameManager.cpp:27-28` , `include/patterns/EventBus.h:36-58` , `src/patterns/EventBus.cpp:196-208` , `include/core/SoundManager.h:116-130` , `src/core/SoundManager.cpp:40-48` ,

include/core/TextureManager.h:23-33 , include/core/TextureManager.h:64-84 , src/core/TextureManager.cpp:15-21 , include/core/SaveManager.h:22-26 .

6. The Command pattern — turning input into swappable requests

Problem and participants

If Game or PlayState wrote if (key X) mario->... for every key, remapping keys or supporting two players would glue input to gameplay. Command wraps a request into an object.

Role	Real class/symbol
Command contract	ICommand::execute() in include/patterns/ICommand.h:16-20
Invoker/registry	InputHandler , mapping keys to vector<Binding> in include/patterns/InputHandler.h:40-80
Concrete commands	JumpCommand , MoveLeftCommand , MoveRightCommand , PauseCommand ; RunCommand / ShootCommand wrap callbacks
Receiver	Mario for move/jump; a PlayState callback invoking Level::requestFireBallShot ; Pause publishes an event
Wiring/client	PlayState::rebindCommands() in src/states/PlayState.cpp:62-134

One-frame workflow

1. Game::processEvents() updates the InputState (Pressed/Held/Released) then GameManager::processInput() forwards that snapshot to the top state.
2. PlayState::processInput() resets intents so stale input does not "leak" across pause or transitions, then calls m_inputHandler.handleInput(inputState) (src/states/PlayState.cpp:416-457).
3. InputHandler checks triggers. For a bound key it calls binding.command->execute() (src/patterns/InputHandler.cpp:34-95). Horizontal / Vertical bindings additionally pick the key with the newest press-order so two directions cannot run at once.
4. JumpCommand /move commands call Mario methods. ShootCommand calls the Level request; Level checks the FIRE state, cooldown, the two-FireBall limit, and may queue while Box2D is locked.
5. After input, Level::update() applies intents to physics/entities. A command never owns or updates physics itself.

Escape is the easy example: PauseCommand::execute() publishes GAME_PAUSED over the EventBus ; PlayState::onNotify() queues GameManager::pushState(PauseState) . The command only describes the request; State/Observer decide how the system reacts.

Benefits, trade-offs, and extension points

- Design benefits: remap keys with bindKey , share actions for co-op, test commands in isolation, and keep input logic out of Game .

- `InputHandler` **owns** commands via `unique_ptr` ; `getAction()` returns only a non-owning pointer. Commands hold a raw `Mario*` /callback, so they must be rebound after every `Level` reload.
- This is not an undo/redo system: `ICommand` currently has only `execute()` — no `undo()` , history, or request queue. `gameplayEnabled=false` drops the current input; it does not buffer it.
- To add an action: create a class implementing `ICommand` (or reuse a callback command), bind it with the proper trigger/group in `rebindCommands()` , and guard against Mario dying, pausing, or transitioning.

Evidence: `include/patterns/ICommand.h:16-20` , `include/patterns/InputHandler.h:49-80` , `src/patterns/InputHandler.cpp:10-95` , `src/patterns/JumpCommand.cpp:15-23` , `src/patterns/PauseCommand.cpp:18-23` .

7. The Observer pattern — one event, many reactions

Problem and participants

When Mario picks up a Coin, gameplay should not know the details of a HUD refresh or which sound file plays. Observer lets a subject notify while many observers register themselves.

Role	Real class/symbol
Subject contract	`ISubject::subscribe/unsubscribe/notify`
Subject/bus	The `EventBus` Singleton, `EventBus::getInstance()`
Observer contract	`IObserver::onNotify(const GameEvent&)`
Concrete observers	`PlayState` , `HUD` , `SoundManager`
Data	`EventType` + value-only `GameEvent` / `EventContext` (`std::variant`)
Lifetime token	move-only `Subscription` ; keeping the token alive means still subscribed

Workflow

1. On entering Play, `PlayState::onEnter()` subscribes to the four events it handles; HUD and SoundManager subscribe to their own event groups. Each call returns a `Subscription` stored in a vector.
2. Gameplay/collision calls `EventBus::notify(...)` . For example `CollisionManager::defeatEnemy()` publishes `ENEMY_STOMPED` or a defeat event; `Mario::powerUp()` publishes `PLAYER_POWER_UP` , `Mario::powerDown()` publishes `PLAYER_POWER_DOWN` when only dropping a form, and `Mario::loseLife()` publishes `PLAYER_DIED` ; Level publishes `LEVEL_COMPLETED` .
3. `EventBus::notify(const GameEvent&)` dispatches **synchronously** to the registered observers (`src/patterns/EventBus.cpp:242-267`). It copies a listener snapshot so a callback may unsubscribe without corrupting the loop, then verifies the lease is still active before each callback.

4. `PlayState::onNotify()` updates progress/queues states; `HUD::onNotify()` refreshes text or timers; `SoundManager::onNotify()` picks an SFX. The publisher needs to know none of these three classes.
5. `onExit()` /destructors clear the tokens. The `Subscription` destructor calls `reset`; the final lease disconnects from EventBus (`include/patterns/Subscription.h:23-48` and `src/patterns/EventBus.cpp:161-194`).

Benefits, trade-offs, and limits

- Benefits: add HUD/audio/analytics observers without touching publishers; the payload is value-only so the bus holds no pointer/reference to domain objects.
- EventBus is a Singleton, which is convenient for game-wide events but creates global coupling; event names/types remain a shared contract.
- Dispatch is synchronous, not a message queue: observers run inside the `notify` call, so long or re-entrant callbacks can affect the frame. Never publish pointers inside `GameEvent`; `EventContext` deliberately accepts only the defined value contexts.
- The raw `IObserver*` in a record does not own the observer. You **must** keep the `Subscription` and destroy the token before/when the observer dies; the token is lifecycle safety, it does not make the observer an owned object.
- `notify(EventType)` is a compatibility overload that builds an empty `GameEvent`; for new data prefer a clearly typed context.

Evidence: `include/patterns/EventBus.h:36-57`, `include/patterns/IObserver.h:14-26`, `include/patterns/GameEvent.h:38-61`, `src/patterns/EventBus.cpp:210-271`, `src/states/PlayState.cpp:281-312`, `src/ui/HUD.cpp:112-146`, `src/core/SoundManager.cpp:45-109`, `src/physics/CollisionManager.cpp:521-594`.

8. The State pattern — swapping the rule set based on the current state

---

The project has **two State layers** that are related but must not be confused:

1. **Game state:** `IGameState` with `MenuState`, `PlayState`, `PauseState`, `GameOverState`, `WinState`, ... representing big screens/flows.
2. **Mario power-up state:** `IMarioState` with `SmallMarioState`, `SuperMarioState`, `SmallFireMarioState`, `SuperFireMarioState`, owned by `Mario` through a `unique_ptr`.

8.1 The game state stack and deferred transitions

`IGameState` defines `onEnter`, `onExit`, `onPause`, `onResume`, `processEvents`, `processInput`, `update`, `render`, `isOverlay`. `GameManager` does not swap objects in the middle of a callback; `changeState`, `pushState`, `popState` only append a `PendingOp`. After `top()->update(dt)`, `processPendingOps()` swaps the queue and applies it at the end of update (`src/core/GameManager.cpp:32-113`).

The concrete Pause flow:

1. `PlayState` is on top and binds Escape to `PauseCommand`.
2. The command publishes `GAME_PAUSED`; `PlayState::onNotify` calls `GameManager::pushState(std::make_unique<PauseState>())`.
3. At the end of update, the manager calls `PlayState::onPause`, pushes Pause, then `PauseState::onEnter`.
4. Because `PauseState::isOverlay()==true`, rendering draws the state below first; input and update go only to Pause. Popping calls `PauseState::onExit`, then Play `onResume`.

`CHANGE` calls `onExit` on every state and replaces the whole stack; `PUSH` keeps the state below as an overlay; `POP` drops the top. Deferral can add latency up to the frame's safe point, but it avoids destroying an object that is on the call stack and avoids mutating the vector while iterating.

8.2 Mario form state

When Mushroom/FireFlower call `Mario::powerUp`, `Mario::applyStateTransition` picks the matching state object, and `Mario` itself rebuilds animation and the fixture. In the current production runtime, `Mario::canBreakBricks()` is the method that delegates through `m_statePattern->canBreakBricks()` (or Star power). By contrast, `Mario::canShootFireBall()` checks the `MarioState` enum and cooldown directly; it does **not** call `m_statePattern->canShootFireBall()`.

`IMarioState` still declares `getStateType()`, `getHitboxSize()`, `canShootFireBall()`, `canBreakBricks()` plus `onEnter/onExit/update`, and the concrete classes implement them. However, a call-site search currently shows production only reads `canBreakBricks()` through the state; there are no production calls to `getHitboxSize()`, `getStateType()`, `canShootFireBall()`, or the `IMarioState` lifecycle/update callbacks. So distinguish the **implemented seam** from the **behavior actually used**. If Box2D is locked or there is insufficient headroom, growth is stored in `m_pendingGrowthState` and processed at a safe update.

Important caveat: this is a real State seam, but not all logic has been pushed into the four state classes. `Mario` still holds the enum, the state transitions, timers, animation setup, the fixture, and much of the movement/damage policy; the states' `update()` is still minimal. So describe it as "Mario has a State seam for forms, of which production currently uses the brick-breaking capability" — not as a promise that each form is an independent gameplay engine.

Evidence: `include/states/IGameState.h:19-35`, `include/core/GameManager.h:26-74`, `src/core/GameManager.cpp:44-113`, `include/states/IMarioState.h:17-33`, `include/entities/Mario.h:54-63`, `include/entities/Mario.h:187-205`, `src/entities/Mario.cpp:963-1015`, `src/entities/Mario.cpp:1076-1161`, `src/entities/Mario.cpp:1486-1493`, `include/states/SmallMarioState.h:10-24`.

9. The Factory Method pattern — creating objects through a creator seam

Problem and participants

The tile map contains codes `G`, `K`, `C`, `?`, ...; if `Level` sprinkled `new Goomba`, `new Koopa`, `new Coin` everywhere, the mapping and constructor context would be scattered. Factory Method centralizes the choice of creator/constructor while still returning the `unique_ptr<Entity>` abstraction.

Role	Real class/symbol
Product abstraction	`Entity`
Creator contract	abstract `EntityCreator::create(const SpawnRequest&, const SpawnContext&)`
Concrete creators	`EnemyCreator`, `ItemCreator`, `WorldObjectCreator`
Orchestrator/canonical seam	`EntityFactory::create(...)`
Request/context	`SpawnRequest` (`std::variant<EnemyType, ItemType, char>`), `SpawnContext` (world/theme)
Client/owner	`Level::spawnEntitiesFromTileMap()`; Level receives ownership

Level-load workflow

1. `Level::spawnEntitiesFromTileMap()` creates an `EntityFactory`, walks `SPAWN_CODES`, converts tiles to world position/theme and builds `SpawnRequest::tile(code, worldPos)` (`src/level/Level.cpp:685-750`).
2. `EntityFactory::create` uses `std::visit` on the payload. Enemies pick `EnemyCreator`, items pick `ItemCreator`, char/tile picks `WorldObjectCreator` (`src/patterns/EntityFactory.cpp:18-34`).
3. The creator maps the enum/tile code to a concrete constructor, passing the Box2D world and theme. `WorldObjectCreator` may delegate to the enemy/item creators; a wrongly typed request or unsupported code returns `nullptr`.
4. When an entity exists, Level sets the `TextureManager`; enemies get the `TileMap` attached; then `m_entities.push_back(std::move(entity))`. From then on Level is the single owner — update/render/remove go through the base interface and the destructor cleans up.

Mario is a deliberate exception: `Level` creates Mario directly at lines 693-702 because Mario has its own lifecycle/player wiring; the factory currently centralizes enemy, item, and world-object spawning from the map.

Canonical seam, compatibility, and limits

- The canonical production API is the **non-static** `EntityFactory::create` and the abstract `EntityCreator` method. The static `createEnemy`, `createItem`, `createFromTileCode` are compatibility shims; they forward to a `defaultFactory()` to avoid duplicating the mapping.

- `EntityFactory` has a public constructor and multiple objects can exist. The static function-local `defaultFactory` does not make `EntityFactory` a Singleton.
- The mapping is still a centralized `switch`. Adding an enemy/item usually means adding an enum value and the corresponding case; that is the explicit trade-off of the `variant` request, not "extension without modifying code".
- Practical benefits: clients need not include/know every concrete entity's constructor, ownership is returned clearly, and creator categories are easy to test/replace.

Evidence: `include/patterns/EntityFactory.h:20-50`, `include/patterns/EntityCreator.h:14-20`, `include/patterns/SpawnRequest.h:44-99`, `src/patterns/EntityFactory.cpp:12-59`, `src/patterns/EnemyCreator.cpp:24-76`, `src/patterns/ItemCreator.cpp:16-36`, `src/patterns/WorldObjectCreator.cpp:31-116`, `src/level/Level.cpp:685-750`.

10. Workflows connecting OOP and the patterns

10.1 One frame: input -> gameplay -> render

1. `Game::run()` computes `dt`, then calls `processEvents`, `update`, `render`.
2. Window events update the `InputState`; `GameManager::getInstance()` forwards to the top `IGameState` (Singleton access + abstraction + runtime polymorphism).
3. `PlayState` resets intents and `InputHandler` dispatches commands. The commands call Mario/Level APIs without poking private physics (encapsulation).
4. `GameManager::update` calls `PlayState::update`; PlayState calls `Level::update`. Level steps Box2D and updates Mario and the Entity list via `Entity::update` (composition + polymorphism).
5. Level/State/HUD render per owner; `GameManager` handles the overlay when the top state is Pause. Pending state transitions apply only at the safe point at the end of update.

10.2 Loading a tile map -> entities alive inside Level

`PlayState::onEnter()` loads the `Level`; Level loads the TileMap/world, creates Mario directly, then the Factory creates enemies/items/world objects. The `unique_ptr` travels from creator -> Level's vector; each frame entities update virtually. Spawners may return pending `unique_ptr<Entity>` via `takePendingSpawns`; Level collects them first and appends afterwards so it never mutates the vector while iterating. When an entity is marked/removed, `remove_if` drops it and RAII destroys the physics/entity.

10.3 Collision -> events -> UI/audio/state

`ContactListener::BeginContact` hands contacts to `CollisionManager::resolve`; the manager builds a `CollisionContext`, calls entity callbacks, and dispatches policy. On stomp/defeat the collision manager publishes an event. Item collisions call `Item::onCollect`;

Mario/Level publish power, coin, fireball, death/completion events. Specifically, `Mario::loseLife()` publishes `PLAYER_DIED`, not `PLAYER_POWER_DOWN`; `powerDown()` publishes `PLAYER_POWER_DOWN` only when dropping a form without losing a life. EventBus dispatches synchronously:

- `HUD::onNotify` updates score/lives/power/timer;
- `SoundManager::onNotify` plays SFX;
- `PlayState::onNotify` shakes the camera, records pending death/reload/game-over, or starts the completion fade. It does not switch to `WinState` or `GameOverState` directly inside the callback.

A publisher may still call direct methods (e.g. Level calling `item->onCollect`), but the global notification part is Observer. Do not label the entire call graph as EventBus.

Direct-damage exception: the Bullet Bill contact policy in `CollisionManager::handleMarioCollision()` calls `Mario::loseLife()` or `queuePowerDown()` straight from the lateral-collision branch (front-nose impact = instant death, rear brush = power-down) without creating a defeat transaction. The Observer flow still starts afterwards, because `loseLife()` itself publishes `PLAYER_DIED`.

10.4 Power-up -> Mario State -> event

The item calls `Mario::powerUp`; Mario validates the upgrade, queues if the world is locked, swaps the `IMarioState`, rebuilds hitbox/animation, then publishes `PLAYER_POWER_UP`. HUD and SoundManager react via Observer. Production uses `Mario::canBreakBricks()` to read the brick-breaking capability from `m_statePattern`, but `Mario::canShootFireBall()` uses the `MarioState` enum and cooldown directly, without reading the state's fire capability callback. On damage, `powerDown` steps `FIRE_SUPER -> SUPER -> SMALL` and publishes `PLAYER_POWER_DOWN`; if already SMALL, `loseLife()` decrements a life then publishes `PLAYER_DIED`.

10.5 Safe screen transitions

There are two terminal flows to keep clearly separated:

1. `Mario::loseLife()` publishes `PLAYER_DIED` at `src/entities/Mario.cpp:1169-1228`. `PlayState::onNotify()` only shakes the camera, saves the score, and sets `m_isReloadPending` or `m_isGameOverPending` (`src/states/PlayState.cpp:349-386`). After the death animation completes or the fallback timer expires, `PlayState::update()` sets `m_needsReload` / `m_needsGameOver` and calls `navigateToLevel()` or queues `changeState(GameOverState)` (`src/states/PlayState.cpp:656-694`).
2. `LEVEL_COMPLETED` reaches `PlayState::onNotify()` to snapshot progress and start the fade (`src/states/PlayState.cpp:391-406`). Inside `updateTransition()`, only passing the final level queues `WinState`; otherwise the next level loads (`src/states/PlayState.cpp:727-732`).

`GameManager` applies these requests only at safe points. The chain illustrates State + Observer + composition ownership + deferred mutation; do not call `changeState` directly from code iterating `m_stateStack`.

11. Extension recipes for developers

Adding a Command

1. Identify the receiver and its lifetime: the current Mario, Level, or a safe callback.
2. Create a class deriving from `ICommand`, implement `execute()`; hold non-owning dependencies or a small callback — never manage the receiver yourself.
3. Bind with a `unique_ptr` inside `PlayState::rebindCommands()` using the right `Pressed/Held/Released` and `InputGroup`.
4. If a Level reload creates a new Mario, call `rebindCommands()` again; never keep the old pointer. Verify the action is blocked during transitions/death/pause.
5. Test rebind/unbind plus Held and Released triggers; remember `InputHandler` does not buffer suppressed input.

Adding an Entity/enemy/item

1. Pick the right relationship: derive from `Enemy` / `Item` when it is that kind of entity; use composition for components/lifetime instead of creating an inheritance branch merely to hold an object.
2. Implement the pure virtual `update`, type/subtype/capability, and the specific contract (`Enemy::patrol/onStomp`, `Item::onCollect`, ...).
3. Choose the request: add an `EnemyType` / `ItemType` enum, or a tile code in `SpawnRequest`; add the mapping in the matching creator. World objects may delegate through `WorldObjectCreator`.
4. Make sure the constructor accepts `SpawnContext.world/theme` when needed; unsupported requests return `nullptr` instead of a half-initialized object.
5. When Level receives the result: set the texture, attach the TileMap for enemies, `std::move` into `m_entities`; do not create a parallel owner. Verify update, removal, Box2D destruction, and rendering.

Adding an event

1. Add the `EventType` and, if data is needed, a value struct inside `EventContext`; never put raw pointers/references/domain objects into the variant.
2. Subscribers derive from `IObserver`, subscribe at a clear lifecycle point and keep the `Subscription` member; clear tokens in `onExit`/destructors.
3. The publisher calls `notify(GameEvent{...})` at the right moment; remember dispatch is synchronous and callbacks may re-enter/unsubscribe.
4. Update each observer that has a reason to receive the event. If there is only a direct caller, do not add EventBus for form's sake.
5. Verify the token is move-only, subscriptions are not duplicated, observers do not die before tokens, and the payload is copyable.

Adding a Game State or Mario State

1. For a screen/game mode: derive from `IGameState`, implement lifecycle + frame methods, choose `isOverlay()`. Store owned resources as smart pointer members or by value.

2. Transition via `GameManager::changeState/pushState/popState` with a `unique_ptr`; never edit the stack yourself or destroy a state inside a callback.
3. For a power-up: implement `IMarioState`, add the branch in `Mario::applyStateTransition`, then audit the **production call sites**. Currently only `Mario::canBreakBricks()` reads a capability from the state; fire eligibility, hitbox, and animation are still coordinated by `Mario`/the enum.
4. `IMarioState` is not `IGameState`: do not add `onPause/onResume`, EventBus subscriptions, or command bindings to Mario states. The `IMarioState::onEnter/onExit/update` callbacks are currently contract without production call sites; to use them you must add a deliberate call site and verify its lifecycle.
5. Verify the right lifecycle for the kind of state added: a game state needs `onEnter/onExit/onPause/onResume`, overlay/input/render and failure paths; a Mario state needs transitions, fixture/animation/ceiling/world-lock handling, and must not keep pointers to the old Level/Mario after a reload.

12. Pitfalls and things not to mislabel as patterns

Misconception	The truth in the code
"`SaveManager` is a Singleton because saves are game-wide."	Wrong. `SaveManager` has a public constructor at `include/core/SaveManager.h:22-26`; `GameManager` holds `SaveManager m_saveManager` by value at `include/core/GameManager.h:72-74`. Use `GameManager::getInstance().getSaveManager()` to reach the owned object; do not add `SaveManager::getInstance()`.
"Every static factory function is a Simple Factory."	Wrong/incomplete. The canonical seam is the non-static `EntityFactory::create -> EntityCreator -> EnemyCreator/ItemCreator/WorldObjectCreator`. Static helpers are only compatibility forwarding (`include/patterns/EntityFactory.h:25-45`, `src/patterns/EntityFactory.cpp:37-59`). Use the name Factory Method for the current implementation.
"`defaultFactory()` makes EntityFactory a Singleton."	No. `EntityFactory` remains public-constructible with member creators; the function-local instance only avoids duplicating the mapping for the shim.
"Inheritance and composition are the same."	`Mario : Character` or `Coin : Item` is inheritance. `Level` holding `unique_ptr<Mario>` / `unique_ptr<Entity>`, or `Mario` holding `unique_ptr<IMarioState>`, is composition + ownership.
"EventBus replaces every direct call."	No. Collision/Level still call direct domain methods; EventBus is for global notification to HUD/Sound/PlayState.
"Observer is an async queue."	No. `notify` invokes `onNotify` immediately in the same call stack; the snapshot only protects iteration/lifecycle.
"State has extracted all Mario logic."	Not yet. `Mario` still holds the enum, transitions, physics, fixture, and animation orchestration; the concrete `IMarioState` classes expose many callbacks/capabilities but production currently reads only `canBreakBricks()` through the state.
"`EntityView` allows mutating entities."	No. It is a read-only, non-owning view; only Level mutates ownership.
"Holding a raw Mario pointer in a command is fine forever."	Only for the current Level's lifetime. Reloads must rebind; tokens/unique_ptr do not fix raw pointers by themselves.

13. Checklist before reviewing a change

- Have I identified whether this is **is-a** (inheritance) or **has-a/owns-a** (composition)?
- Does the new interface have a small, clear pure-virtual contract with real users?
- Which object owns the resource? Are there `unique_ptr`, values, or RAII tokens with the right lifetime? Does any raw pointer outlive its owner?
- Does the change go through existing seams (`ICommand`, `IObserver`, `IGameState`, `EntityCreator`) instead of adding global `if/switch` in clients?
- If it is a Command: is it rebound after Level reloads, and blocked in non-functional states?

- If it is an event: is the payload value-only, is the token kept and cleared at the right time, and is the synchronous callback safe?
- If it is a **game state**: is the operation deferred through GameManager; are the overlay lifecycle and failure paths clear? For a Mario State, follow the dedicated recipe above for transitions and physics/world-lock.
- If a Singleton is used: does the source prove `getInstance()`, the constructor/copy policy, and a legitimate reason for global access? Have you weighed global coupling, testability, and avoiding turning `SaveManager` into a Singleton?
- If it is an entity: are the factory mapping, world/theme context, texture, removal, and the owning `Level` all wired up?
- Am I describing **evidence** or merely claiming a design benefit/inference? If it is an inference, are its limits stated?

14. Quick source map

Topic	Read first
Entity hierarchy, virtual contract, capability	`include/entities/Entity.h:31-42` ; `include/entities/Entity.h:116-135` ; `include/entities/Character.h:17-31` ; `include/entities/Enemy.h:21-42` ; `include/items/Item.h:14-42`
Level/entity ownership	`include/level/Level.h:210-218` ; `src/level/Level.cpp:685-750` ; `src/level/Level.cpp:1102-1238` ; `src/level/Level.cpp:1174-1182`
Game loop/State manager	`src/core/Game.cpp:78-103` ; `include/core/GameManager.h:26-74` ; `src/core/GameManager.cpp:32-113`
Singleton services	`include/core/GameManager.h:19-50` ; `include/patterns/EventBus.h:36-58` ; `include/core/SoundManager.h:116-130` ; `include/core/TextureManager.h:23-33` ; `include/core/TextureManager.h:64-84` ; `src/core/TextureManager.cpp:15-21`
Command input	`include/patterns/ICommand.h:16-20` ; `include/patterns/InputHandler.h:40-80` ; `src/patterns/InputHandler.cpp:10-95` ; `src/states/PlayState.cpp:416-457`
Observer/event lifetime	`include/patterns/EventBus.h:36-57` ; `include/patterns/Subscription.h:23-48` ; `src/patterns/EventBus.cpp:210-271`
Game/Mario State	`include/states/IGameState.h:19-35` ; `include/states/IMarioState.h:17-33` ; `src/entities/Mario.cpp:963-1015`
Factory Method	`include/patterns/EntityFactory.h:20-50` ; `include/patterns/EntityCreator.h:14-20` ; `src/patterns/EntityFactory.cpp:18-59` ; `src/level/Level.cpp:685-750`
Collision -> event	`src/physics/ContactListener.cpp:14-25` ; `src/physics/CollisionManager.cpp:724-763` ; `src/physics/CollisionManager.cpp:521-594`
Save ownership correction	`include/core/SaveManager.h:22-40` ; `include/core/GameManager.h:42-42` ; `include/core/GameManager.h:72-74`

Short glossary

- **Abstraction:** a contract that keeps what the client needs to use.
- **Encapsulation:** hiding state/invariants, accessed through methods.
- **Inheritance:** an "is-a" relationship; the subclass receives the base contract.
- **Polymorphism:** calling the same base API while the real implementation runs.
- **Composition:** an object containing other objects; usually paired with ownership.
- **Singleton:** a class with one globally accessible instance; use deliberately because dependencies are hidden and tests struggle to replace

the service.

- **Factory Method:** a creator abstraction that chooses and builds the concrete product.
- **Command:** a request wrapped into an object with `execute()`.
- **Observer:** a subject publishing notifications to many registered observers.
- **State:** the current object changing how the context behaves/lives.
- **RAII:** an object's lifetime managing the resource/token.
- **Non-owning:** a pointer/reference that merely observes; it must never destroy the object.
- **Deferred operation:** record the request now, apply it at a defined safe point.